

and change properties. You can refer to Figure 1 for an example of the desired layout. The R1C1 notation used in the following instructions indicates which cell of the table to add a control to; for example, R1C2 means, "...the cell where the second column intersects the first row." Perform the following steps:

1. Add a *Table* container into the first page of the notebook (we use it to keep the other, contained, controls aligned). In the pop-up dialogue that appears when you add the table, set it to have four rows and four columns.
 2. Place a label at R1C2 of the table, and change its *Label* property to 'Welcome'.
 3. Place a label at R2C2, and change its *Label* property to 'Login'.
 4. Place a label at R3C2, and change its *Label* property to 'Password'.
 5. Place a text entry widget at R2C3 and change its *Name* to 'pw_ent'.
 6. Place a text entry widget at R3C3 and change its *Name* to 'pw_ent'.
 7. Place a button at R4C2. Change its *Label* to '_OK, its *Use underline* to Yes, and its *Name* to 'ok_btn'.
 8. Place a button at R4C3. Change its *Label* to '_Cancel', its *Use underline* to Yes, and its *Name* to 'cancel_btn'.
 9. Double-click the notebook's second page. In the empty area of the page, place a single label widget. Change its *Label* property to 'Authentication succeeded! You can now close the window.' (This is a place-holder label. We will design the 'employee information viewer' screen in the second notebook page in a later article in this series, and we will delete this place-holder label at that time.)
 10. Double-click the notebook's first page to make it the active page again.
 11. In the Glade object tree pane, find and select the *notebook1* widget. In its *General* properties, toggle the *Show Tabs* property to No. This will prevent users from clicking the tab for the second page, and accessing the 'employee information viewer' screen without first logging in.
 12. An important requirement for our login screen is that the *pw_ent* text entry should not display the text of the password to a casual onlooker. To mask the password as the user types it in, select the text entry, and toggle the *Invisible Character Set* property (in *General* properties) to Yes. Glade, by default, provides a dot-like masking character, in the *Invisible character* property, which will replace each character in this text entry. If you like, you can change this to another character, such as an asterisk (*) or another non-alpha-numeric character of your choice. Avoid using a space or other 'invisible' character, though; it can confuse users, who will get no visible feedback in response to their typing in the password entry.
- Our login screen design is done; we leave the rest of

the cells in the 4x4 table empty. Save the file, in Glade, supplying the name '*employee-builder.glade*'. This file is an XML file in the new GtkBuilder format that is built into newer versions of GTK. After setting up the required signal handlers, we will save this file a final time, and proceed to writing the code that turns our design into a running application window. Before that, let's get some understanding of the signals.

Events, signals and GTK+ programming

The programming model that GTK+ provides is event-driven, like many other windowing tool kits. That is, the flow of the program execution is controlled by events like user inputs, messages from other programs, or some other kind of external data. For GTK+, almost all events are in the form of messages from the underlying X server. The main loop (*Gtk2->main()*) loops, waiting for events. When there is an event (say, a mouse click), the main loop wakes up and delivers the event to one or more widgets. When widgets receive events, they emit signals. Signal emission is the process of GTK running all handlers for a specific control object (widget) and the emitted signal. Handler functions (also called callbacks) can be bound to signals using the *g_signal_connect()* function, or similar ones. The signal-handling callback function can be written to effectuate the desired behaviour of the application in response to the event received. After the handler finishes, control is generally transferred back to the main loop.

It is not mandatory to implement handlers for every signal for every widget. Almost all signals are handled by default handlers. A widget inherits its parent's default signal handlers. When a signal is emitted by a widget, first the signal ID is looked up, and all normally connected callbacks are invoked before the default handlers. If one signal callback function needs to be executed after the default handler, it must be connected using a different function (*g_signal_connect_after()*). This process of signal emission is repeated up the widget hierarchy, till the top-level widget is reached, or propagation is stopped by a handler (by returning *TRUE*).

Event handling is a complex subject. GTK+ uses GDK (the GIMP Drawing Kit) in order to abstract the underlying graphical windowing system. This makes it easier to port GTK+ to various platforms. For example, with Linux, GDK uses the XLib library by X Windowing System, whereas with Microsoft Windows, GDK uses Windows' own windowing system libraries. In general, a developer need not be concerned about GDK, unless a new widget needs to be designed.

Finally, GTK+ signals and POSIX signals have nothing in common, except the confusing name.

Let's add a signal handler to the top-level window:

1. Click on the toplevel window (*window1*)
2. Click the *Properties* pane and choose the *Signals* tab.